

Machine 类



Machine类

src/include/Machine.h

```
class Machine
{
    .....
private:
    static Machine instance;    /* Machine单体类实例 */

    IDT* m_IDT;                 // 中断向量表
    GDT* m_GDT;                 // 内存描述符表
    PageDirectory* m_PageDirectory;    // 系统页目录 0xC02000000
    PageTable* m_KernelPageTable;    // 系统，核心页表 0xC02001000
    PageTable* m_UserPageTable;    // 系统，用户页表 0xC02002000
    TaskStateSegment* m_TaskStateSegment; // TSS结构
};
```



```
class Machine
{
    /* static const member */
public:
    .....

    /* 段寄存器的值：5个段选择子、内核数据段，用户代码段、用户数据段，TSS段的选择子 */
    static const unsigned int KERNEL_CODE_SEGMENT_SELECTOR = 0x08; // 内核代码段
    static const unsigned int KERNEL_DATA_SEGMENT_SELECTOR = 0x10; // 内核数据段
    static const unsigned int USER_CODE_SEGMENT_SELECTOR = (0x18 | 0x3); // 用户代码段
    static const unsigned int USER_DATA_SEGMENT_SELECTOR = (0x20 | 0x3); // 用户数据段
    static const unsigned int TASK_STATE_SEGMENT_SELECTOR = 0x28; // TSS结构
    static const unsigned int TASK_STATE_SEGMENT_IDX = 0x5; /* TSS段描述符在GDT中的位置 */

    /* 页目录、核心态页表、用户态页表在物理内存中的起始地址 */
    static const unsigned long PAGE_DIRECTORY_BASE_ADDRESS = 0x200000;
    static const unsigned long KERNEL_PAGE_TABLE_BASE_ADDRESS = 0x201000;
    static const unsigned long USER_PAGE_TABLE_BASE_ADDRESS = 0x202000;
    static const unsigned long USER_PAGE_TABLE_CNT = 2;

    static const unsigned int KERNEL_SPACE_SIZE = 0x400000;
    static const unsigned long KERNEL_SPACE_START_ADDRESS = 0xC0000000;

    .....
};
```



```
class Machine
{
    .....

public:
    static Machine& Instance(); /* 返回单态类的instance */

    // 系统初启时，初始化CPU硬件

    void InitIDT(); /* 初始化中断向量表 */
    void InitGDT(); /* 初始化GDT */
    void InitPageDirectory(); /* 初始化页目录 */
    void InitUserPageTable(); /* 初始化核心页表 */
    void InitTaskStateSegment(); /* 初始化TSS结构 */
    void EnablePageProtection(); /* 开启保护模式 */

    void LoadIDT(); /* 把建立好的中断向量表的地址和长度加载进IDTR寄存器 */
    void LoadGDT(); /* 把建立好的GDT表的地址和长度加载进IDTR寄存器 */
    void LoadTaskRegister(); /* 加载TSS结构 */

    .....

};
```



```
class Machine
{
    .....

public:

    // 系统正常运行阶段，定位CPU硬件运行所需的表格。。。
    IDT& GetIDT();                /* get IDT */
    GDT& GetGDT();                /* get GDT */
    PageDirectory& GetPageDirectory(); /* get 内核写系统页目录所需的指针 */
    PageTable& GetKernelPageTable(); /* get 系统核心页表，1张页表，1024个PTE*/
    PageTable* GetUserPageTableArray(); /* get 系统用户页表，2张页表，每张1024个PTE*/
    TaskStateSegment& GetTaskStateSegment();

    .....
};
```

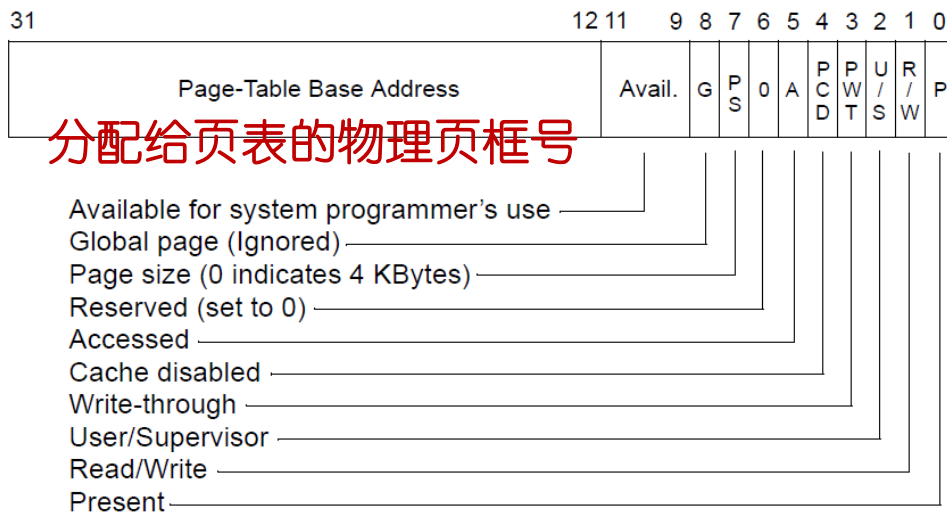
页目录 (Page Directory)

```
class PageDirectory
{
    .....
public:
    PageDirectoryEntry m_Entrys[1024];
};
```

```
struct PageDirectoryEntry // PDE
{
```

```
    unsigned char    m_Present : 1;
    unsigned char    m_ReadWriter : 1;
    unsigned char    m_UserSupervisor : 1;
    unsigned char    m_WriteThrough : 1;
    unsigned char    m_CacheDisabled : 1;
    unsigned char    m_Accessed : 1;
    unsigned char    m_Reserved : 1;
    unsigned char    m_PageSize : 1;
    unsigned char    m_GlobalPage : 1;
    unsigned char    m_ForSystemUser : 3;
    unsigned int      m_PageTableBaseAddress : 20;
}__attribute__((packed));
```

Page-Directory Entry (4-KByte Page Table)



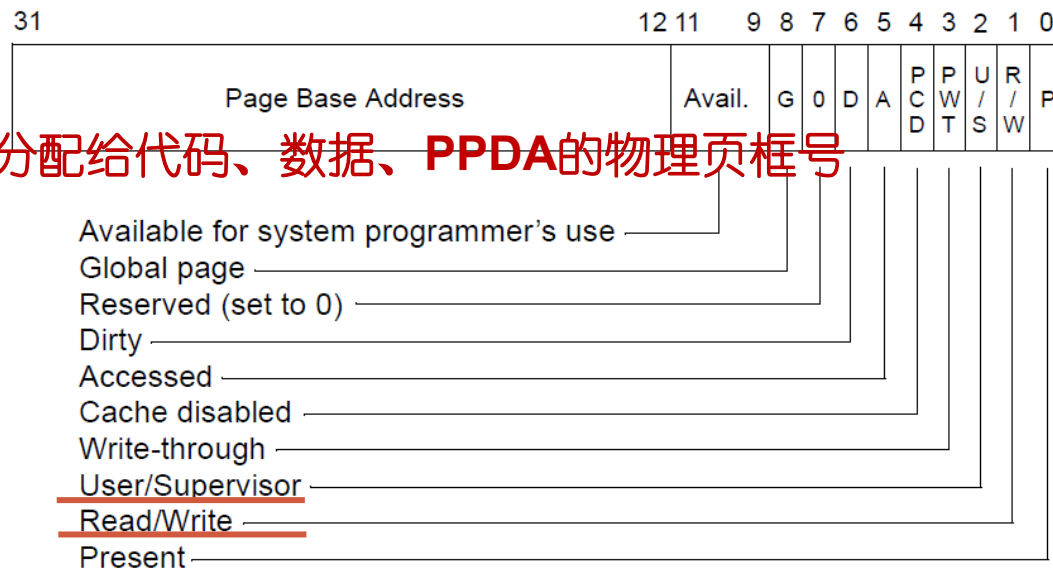
页表 (Page Table)

```
class PageTable
{
    .....
public:
    PageTableEntry m_Entrys[1024];
};
```

```
struct PageTableEntry
{
```

```
    unsigned char    m_Present : 1;
    unsigned char    m_ReadWriter : 1;
    unsigned char    m_UserSupervisor : 1;
    unsigned char    m_WriteThrough : 1;
    unsigned char    m_CacheDisabled : 1;
    unsigned char    m_Accessed : 1;
    unsigned char    m_Dirty : 1;
    unsigned char    m_PageTableAttributeIndex : 1;
    unsigned char    m_GlobalPage : 1;
    unsigned char    m_ForSystemUser : 3;
    unsigned int      m_PageBaseAddress : 20;
}__attribute__((packed));
```

Page-Table Entry (4-KByte Page)



分配给代码、数据、PPDA的物理页框号



```
void MemoryDescriptor::MapToPageTable()
{
```

```
    User& u = Kernel::Instance().GetUser();    // 1、找到新选中进程的user结构
```

```
    if(u.u_MemoryDescriptor.m_UserPageTableArray == NULL)    // 2、新选中进程的相对表
        return;
```

// 3、系统用户页表 0xC0202000

```
    PageTable* pUserPageTable = Machine::Instance().GetUserPageTableArray();
```

```
    unsigned int textPF = 0;
```

```
    if ( u.u_procp->p_textp != NULL )
    {
```

```
        textPF = u.u_procp->p_textp->x_caddr >> 12;
```

```
    }    // 4、代码段起始页框号
```

```
    unsigned int pAddrPF = u.u_procp->p_addr >> 12;    // 4、可交换部分起始页框号
```

```
class Machine
{
private:
    .....
    static Machine instance;    /* Machine单体类实例 */

    IDT* m_IDT;                // 中断向量表
    GDT* m_GDT;                // 内存描述符表
    PageDirectory* m_PageDirectory;    // 系统页目录 0xC02000000
    PageTable* m_KernelPageTable;    // 系统，核心页表 0xC02001000
    PageTable* m_UserPageTable;    // 系统，用户页表 0xC02002000
    TaskStateSegment* m_TaskStateSegment;    // TSS结构
};
```



```
void MemoryDescriptor::MapToPageTable()
{
    .....
    for (unsigned int i = 0; i < Machine::USER_PAGE_TABLE_CNT; i++)    // 2张页表
    {
        for ( unsigned int j = 0; j < PageTable::ENTRY_CNT_PER_PAGETABLE; j++ ) // 每张页表, PTE
        {
            pUserPageTable[i].m_Entrys[j].m_Present = 0; // 清0, 擦掉下台进程该逻辑页的映射关系

            if ( 1 == this->m_UserPageTableArray[i].m_Entrys[j].m_Present ) // 若, 上台进程有这页
            {
                if ( 0 == this->m_UserPageTableArray[i].m_Entrys[j].m_ReadWriter )    // RO
                {
                    pUserPageTable[i].m_Entrys[j].m_Present = 1;
                    pUserPageTable[i].m_Entrys[j].m_ReadWriter = 0;
                    pUserPageTable[i].m_Entrys[j].m_PageBaseAddress =
                        this->m_UserPageTableArray[i].m_Entrys[j].m_PageBaseAddress + textPF;
                }
                else if ( 1 == this->m_UserPageTableArray[i].m_Entrys[j].m_ReadWriter )    // RW
                {
                    pUserPageTable[i].m_Entrys[j].m_Present = 1;
                    pUserPageTable[i].m_Entrys[j].m_ReadWriter = 1;
                    pUserPageTable[i].m_Entrys[j].m_PageBaseAddress =
                        this->m_UserPageTableArray[i].m_Entrys[j].m_PageBaseAddress+pAddrPF;
                }
            }
        }
    }
    .....
}
```



```
void MemoryDescriptor::MapToPageTable()  
{  
    .....
```

```
    pUserPageTable[0].m_Entrys[0].m_Present = 1;
```

```
    pUserPageTable[0].m_Entrys[0].m_ReadWriter = 1;
```

```
    pUserPageTable[0].m_Entrys[0].m_PageBaseAddress = 0;    // 让0#逻辑页映射0#物理页框
```

0#物理页框里有应用程序主函数main的启动函数

```
    FlushPageDirectory();
```

```
}
```